Optimizing Qwen2.5-Coder Inference on Tenstorrent Blackhole: Project Checkpoint

Rassul Magauin
Assylzhan Khamiyev
Mohammed Rashed Ali Yahmoor Alshehhi
Sultan Akimaliyev
Hamad Khalifa Alyahyaee
Mohamed bin Zayed University of Artificial Intelligence
Abu Dhabi, UAE

Abstract

This checkpoint describes our progress on optimizing Qwen2.5-Coder inference for the Tenstorrent Blackhole p150 using the tt-metal SDK. We began by profiling the existing `tt_transformers` port of Qwen2.5 with the stock Blackhole tuning recipe on a four-device tensor-parallel mesh. Two things surprised us. First, the single largest kernel in the profile is TopK at 45–60% of per-token device kernel time, but it only runs on one of the 130 Tensix cores. After reading the tt-metal source, the cause turned out to be a power-of-two width requirement in the multi-core TopK path, which our 38016-wide per-device vocab does not satisfy. Second, the fused SDPA kernel is already very cheap (under 1.5%), which tells us attention is not where we should be spending effort. Once TopK is set aside, the MLP feed-forward block is the real hot path at about 31% of kernel time, with the gate/up and down projections doing most of the work. Based on this, we plan three changes: pad the per-device vocab to the next power of two so the existing multi-core TopK kernel actually fires, drop the MLP matmuls from HiFi4 to HiFi2 math fidelity for a 2x matmul throughput win, and fuse the gate and up projections into one wide matmul. We report baseline numbers, the verified status of each target, and the roadblocks we ran into along the way.

1 Introduction and Goals

Large language models for code, such as Qwen2.5-Coder [1], are increasingly being targeted at non-GPU accelerators, and our course project picks one such pairing: Qwen2.5-Coder on Tenstorrent's Blackhole p150. Blackhole is a RISC-V-based tensor processor with 130 worker Tensix cores. Each core has a matrix unit (FPU), a vector unit (SFPU), five Baby RISC-V CPUs, and 1.5 MB of L1 SRAM, all wired together with a 2D NoC [2]. Tenstorrent's programming model, `tt-metal`, lets the developer reach all the way down to kernels, circular buffers, and NoC scheduling. That gives us a lot of flexibility, and also a lot of room to make things slower than they should be.

Direction. Our original plan was to port Qwen2.5-Coder to Blackhole from scratch. After talking with our TA we switched to Direction B (optimization), because the in-tree `tt_transformers` framework already supports the Qwen2.5 family. The new goal is to profile, rank, and then fix whichever kernels are actually slow on Blackhole.

What this checkpoint covers. We set up a profiling pipeline for `tt_transformers` decode on a four-device Blackhole mesh, using the Tracy profiler together with the `tt-perf-report` utility our TA recommended. We then used that pipeline to produce a per-operator breakdown of Qwen2.5-Coder-7B-Instruct decode against the stock tuned recipe, and we used the breakdown to decide what to work on next. Three optimizations came out of this analysis, and we describe them in Section 6.

2 Background

Blackhole p150. The compute grid on Blackhole is 8×10 (130 worker cores visible to `tt-metal`), compared to Wormhole's 8×8. Each core has 1.5 MB of L1 SRAM, and there are eight DRAM banks split across the chip. The in-tree `tt_transformers` code has several Blackhole-specific branches gated on `is_blackhole()`. The most visible ones are a 512-token prefill-length cutoff (Wormhole uses 1024) and a Blackhole-only DRAM weight prefetcher that is currently wired up only for Llama-3.1-8B.

Qwen2.5-Coder-7B-Instruct. We checked the Hugging Face configs and found that Qwen2.5-Coder-7B shares its base transformer with Qwen2.5-7B-Instruct. Both have 28 layers, hidden size 3584, MLP intermediate size 18944, 28 query heads with 4 grouped key/value heads, and `rope_theta` = 10^6. This matters for us because `tt-metal` already ships a tuned performance decoder config for Qwen2.5-7B, and we can reuse it directly.

Fusion in tt-metal. tt-metal does not fuse kernels by writing one monolithic kernel. A reader kernel streams tiles into L1 circular buffers, a compute kernel drains them through the FPU/SFPU, and a writer kernel streams the results out. "Fusion" is really a property of how those buffers are wired up. We lean on five patterns that already exist in the repo: fused SDPA, matmul+bias+activation, distributed RMSNorm, asynchronous AllGather+matmul, and the paged KV-cache update.

3 Methodology

Hardware. One machine with four Blackhole p150 devices at /dev/tenstorrent/{0..3}, running a tt-metal build with ENABLE_TRACY=ON.

Workload. We drive the canonical simple_text_demo.py benchmark from tt_transformers in decode mode: batch 1, 1024-token context, 10 transformer layers, paged attention on, and the stock performance decoder config. The blackhole-4 pytest fixture picks a 4-way tensor-parallel mesh for us. Weights are local bf16 safetensors downloaded from Hugging Face.

Profiling. We wrap the pytest invocation in python -m tracy with the device_kernel_duration analysis turned on. That writes a per-op CSV under the profiler reports directory (ops_perf_results_*.csv), which for our 10-layer run contained 4,916 rows across two trace-replay sessions (compile phase excluded). We process the CSV two ways. First, a small pandas script for per-op aggregates. Second, tt-perf-report v1.2.2 [3], which gives us a stacked per-op-class view and, more usefully for us, math-fidelity annotations.

4 Baseline Results

Table 1 shows the per-iteration device-0 kernel time for Qwen2.5-Coder-7B-Instruct decode on the four-way Blackhole mesh, taken from steady-state trace replay with the compile phase excluded. Per-iteration device kernel time is 12.1 ms for the 7B-Instruct run and 10.4 ms for the Coder-7B rerun. The two runs have bit-identical op counts and shapes, and the TopK kernel time matches to the nanosecond, so the gap is almost certainly compile-cache warm-up between two back-to-back runs. We will redo this with a cold cache before the final report.

5 Bottleneck Analysis

TopK is a sampling-stage dispatch problem, not a layer bug. TopKDeviceOperation takes 45–60% of per-token device kernel time, with input shape $[1, 1, 32, 38016]$ and k=32. What makes this strange is that the kernel only runs on a single Tensix core, out of the 130 that Blackhole exposes. After reading the source, we found out why: a multi-core TopK kernel does exist (under ttnn/.../reduction/topk/), but its dispatch logic in topk_device_operation.cpp requires the input width to be a power of two, because the underlying compute kernel uses a Batcher bitonic sort. Our per-device vocab width is 38016 = 152064/4 (Qwen2.5's vocab split over the 4-way TP mesh), which is not a power of two, so the dispatcher silently falls back to the single-core path. This is not a transformer-layer issue, and not really a kernel bug either. It is a vocab-shape mismatch with the multi-core fast path.

Table 1. Per-iteration device-0 kernel time, Qwen2.5-Coder-7B-Instruct decode, 4-way Blackhole TP mesh, 10 layers, stock tuned recipe. TopK is a sampling-stage op, not a transformer-layer op.

Operator / group	Time (ms)	% of total
TopK (sampling, 1-core)	5.48	45.3
MLP gate+up projections	2.41	19.9
MLP down projection	1.40	11.5
Attention QKV projections	1.03	8.5
CCL (AllGather + ReduceScatter)	0.69	5.7
Layout/tiling/eltwise glue	0.46	3.8
Attention output projection	0.28	2.3
Fused SDPA decode + RoPE	0.15	1.3
RMSNorm	0.13	1.1
Other	0.08	0.6
Total per iter.	**12.11**	**100.0**

The MLP block is where the real time goes. Once TopK is set aside, the MLP feed-forward block is the hot path: about 31% of kernel time per iteration, or roughly 3.81 ms. It splits into the gate and up projections (3584 → 5120, 20 calls per iteration) and the down projection (5120 → 3584). The single slowest op in the whole profile is the down projection at 139.8 μs peak. tt-metal's model config file shows these matmuls running at HiFi4 math fidelity on BF16/BFP8 operands.

Attention is already fused and cheap. The fused SDPA-decode kernel, the head reshapes, RoPE, and the KV-cache update together add up to only 1.3% of per-iter kernel time. We take this as a good sign: the flash-attention-style CB choreography already shipped under ttnn/.../transformer/sdpa/ is doing its job. We do not plan to touch attention.

Tensor-parallel overhead is small but present. AllGather plus ReduceScatter together add up to 5.7%. This is headroom we could claim later, but only after the MLP wins are in.

6 Planned Optimizations

Pad the vocab so multi-core TopK can fire. The multi-core TopK kernel already exists. To make our shape eligible, we pad the per-device vocab from 38016 up to the next power of two (65536) inside the sampling step, run multi-core TopK on the padded buffer, and discard the dummy entries afterward. This is the same workaround the upstream tt-metal team has applied for other models in recent commits, so the precedent is set. The change is small, lives entirely in the sampling code, and does not require touching any kernel. Because sampling runs once per generated token, the speedup applies to decode regardless of how many layers the model has. The main risk is that the merge step in multi-core TopK becomes the new bottleneck. If that happens, our fallback is to drop k from 32 to 16, which halves the gather volume.

HiFi4 → HiFi2 for the MLP matmuls. Blackhole's FPU has a HiFi2 mode that trades the lowest activation bit for about a 2× matmul throughput. `tt-perf-report` flagged this as the biggest single-line change available against the stock recipe. We expect it to cut the MLP slice by up to half, which is around 1.9 ms/iter, or roughly 15% of the TopK-excluded kernel budget. The change is one field in the performance decoder config. The main risk is an accuracy regression, so we will run the in-tree `ci-token-matching` parameterization against a HiFi4 reference to verify that output tokens still match.

Fuse the MLP gate and up projections. We checked `tt_transformers/tt/mlp.py` and confirmed that for Qwen2.5 the gate (`w1`) and up (`w3`) projections are still issued as two separate `ttnn.linear` calls per layer. Our profile agrees: 20 gate+up matmuls across 10 layers, two per layer. Interestingly, the framework already has the scaffolding for the fused form: `load_checkpoints.py` defines a `fuse_mlp_meta` helper that concatenates the two weight tensors into a single `w1_w3`, and the Llama-Meta path uses it. It is just not wired up for Qwen. Our plan is to (i) concatenate the gate and up weights at load time using the same helper, (ii) modify the Qwen MLP class to load `w1_w3` instead of `w1` and `w3`, and (iii) replace the two `ttnn.linear` calls with a single $3584 \rightarrow 10240$ matmul whose output we split in half before applying SwiGLU. This halves dispatch overhead and removes one round-trip through activation memory.

7 Roadblocks and Open Questions

No `tt-smi` on the box. `tt-smi` is not installed on the machine we have access to, so we read device health off the runtime logs `tt-metal` prints on first program launch. This works but is awkward. Installing `tt-smi` is on our list.

Compile-cache variance. The two back-to-back runs of architecturally identical models showed about a 44% difference in Matmul mean time per op. We are pretty sure this is compile-cache state, since op counts and shapes are identical, but we want to confirm it. For every optimization experiment going forward we will clear the cache and report matched-cache deltas.

HiFi2 accuracy. HiFi2 drops the lowest activation mantissa bit. We have not yet confirmed that this leaves generation quality intact on code prompts. We will use `ci-token-matching` and also add a short code-completion sanity check.

Coder vs. Instruct. Because Qwen2.5-Coder-7B is architecturally identical to Qwen2.5-7B-Instruct, we cannot really demonstrate "Coder-ness" at the kernel level. For the final report we plan to include a small code-generation prompt set so that we can show we are actually running the Coder weights, and not just the generic 7B-Instruct model.

8 Next Steps

Between now and the final report, our plan is to land the TopK multi-core fix and re-profile, then measure the HiFi2 MLP speedup and check the token-match rate, then look at gate+up fusion. Two more things are on the list but have not been started yet: a single-device (non-TP) run so we can separate out the CCL overhead from the rest of the numbers, and a prefill-mode profile, since Blackhole's 512-token prefill cutoff probably exposes a different set of hot paths than what we see in decode.

References

[1] Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, Kai Dang, Yang Fan, Yichang Zhang, An Yang, Rui Men, Fei Huang, Bo Zheng, Yibo Miao, Shanghaoran Quan, Yunlong Feng, Xingzhang Ren, Xuancheng Ren, Jingren Zhou, and Junyang Lin. 2024. Qwen2.5-Coder Technical Report. arXiv:2409.12186 [cs.CL]

[2] Tenstorrent. 2026. TT-Metalium: Tenstorrent's Low-Level Programming Model and SDK. https://github.com/tenstorrent/tt-metal. Accessed April 2026.

[3] Tenstorrent. 2026. tt-perf-report: Performance Reporting Utility for TT-Metal Device Kernels. https://github.com/tenstorrent/tt-perf-report. Version 1.2.2, accessed April 2026.